

LAB 08:

TASK 1:

```
/* USER CODE BEGIN PD */
```

```
/* — Gyroscope Register Addresses ————— */
```

```
#define GYRO_WHO_AM_I_ADDR 0x0F /* Task 1: WHO_AM_I register */
```

```
#define I3G4250D_WHO_AM_I 0xD3 /* Task 1: Expected ID - blue board */
```

```
#define L3GD20_WHO_AM_I 0xD4 /* Task 1: Expected ID - green board */
```

```
#define CTRL_REG1 0x20
```

```
#define CTRL_REG1_VAL 0x8F /* PD=1, ODR=400Hz, Zen=Yen=Xen=1 */
```

```
#define CTRL_REG4 0x23
```

```
#define CTRL_REG4_VAL 0x00 /* FS = ±245 dps (default) */
```

```
#define OUT_TEMP 0x26 /* Temperature output register */
```

```
#define OUT_X_L 0x28 /* X-axis low byte (auto-inc from here)*/
```

```
/* — SPI Command Bits ————— */
```

```
#define SPI_READ 0x80 /* Bit7=1 => read */
```

```
#define SPI_AUTO_INC 0x40 /* Bit6=1 => auto-increment address */
```

```
/* — Chip Select Pin ————— */
```

```
#define GYRO_CS_PIN GPIO_PIN_3
```

```
#define GYRO_CS_PORT GPIOE
```

```
/* USER CODE END PD */
```

```
/* USER CODE BEGIN PM */
```

```
#define GYRO_CS_LOW() HAL_GPIO_WritePin(GYRO_CS_PORT, GYRO_CS_PIN, GPIO_PIN_RESET)
```

```
#define GYRO_CS_HIGH() HAL_GPIO_WritePin(GYRO_CS_PORT, GYRO_CS_PIN, GPIO_PIN_SET)
```

```
/* USER CODE END PM */
```

```
I2C_HandleTypeDef hi2c1;
```

```
SPI_HandleTypeDef hspi1;
```

```
UART_HandleTypeDef huart2;
```

```
PCD_HandleTypeDef hpcd_USB_FS;
```

```
/* USER CODE BEGIN PV */
```

```
static uint8_t Gyro_ReadRegister(uint8_t reg_addr);
```

Usage in USER CODE BEGIN 2:

```
uint8_t id = Gyro_ReadRegister(GYRO_WHO_AM_I_ADDR);
```

```
sprintf(uart_buf, "WHO_AM_I = 0x%02X\r\n", id);
```

```
HAL_UART_Transmit(&huart2, (uint8_t*)uart_buf, strlen(uart_buf), 100);
```

```
// static uint8_t Gyro_ReadRegister(uint8_t reg_addr)
```

```
//{
```

```
//  uint8_t cmd   = SPI_READ | (reg_addr & 0x3F);
```

```
//  uint8_t rx_data = 0x00;
```

```
//  GYRO_CS_LOW();
```

```
//  HAL_SPI_Transmit(&hspi1, &cmd, 1, 100);
```

```
//  HAL_SPI_Receive(&hspi1, &rx_data, 1, 100);
```

```
//  GYRO_CS_HIGH();
```

```
//  return rx_data;
```

```
//}
```

```
// — Task 1 (commented out) —————
```

```
// uint8_t who_am_i = Gyro_ReadRegister(GYRO_WHO_AM_I_ADDR);
```

```
// if (who_am_i == I3G4250D_WHO_AM_I)
```

```
//   sprintf(uart_buf, "WHO_AM_I = 0x%02X --> I3G4250D (PASS)\r\n", who_am_i);
```

```
// else if (who_am_i == L3GD20_WHO_AM_I)
```

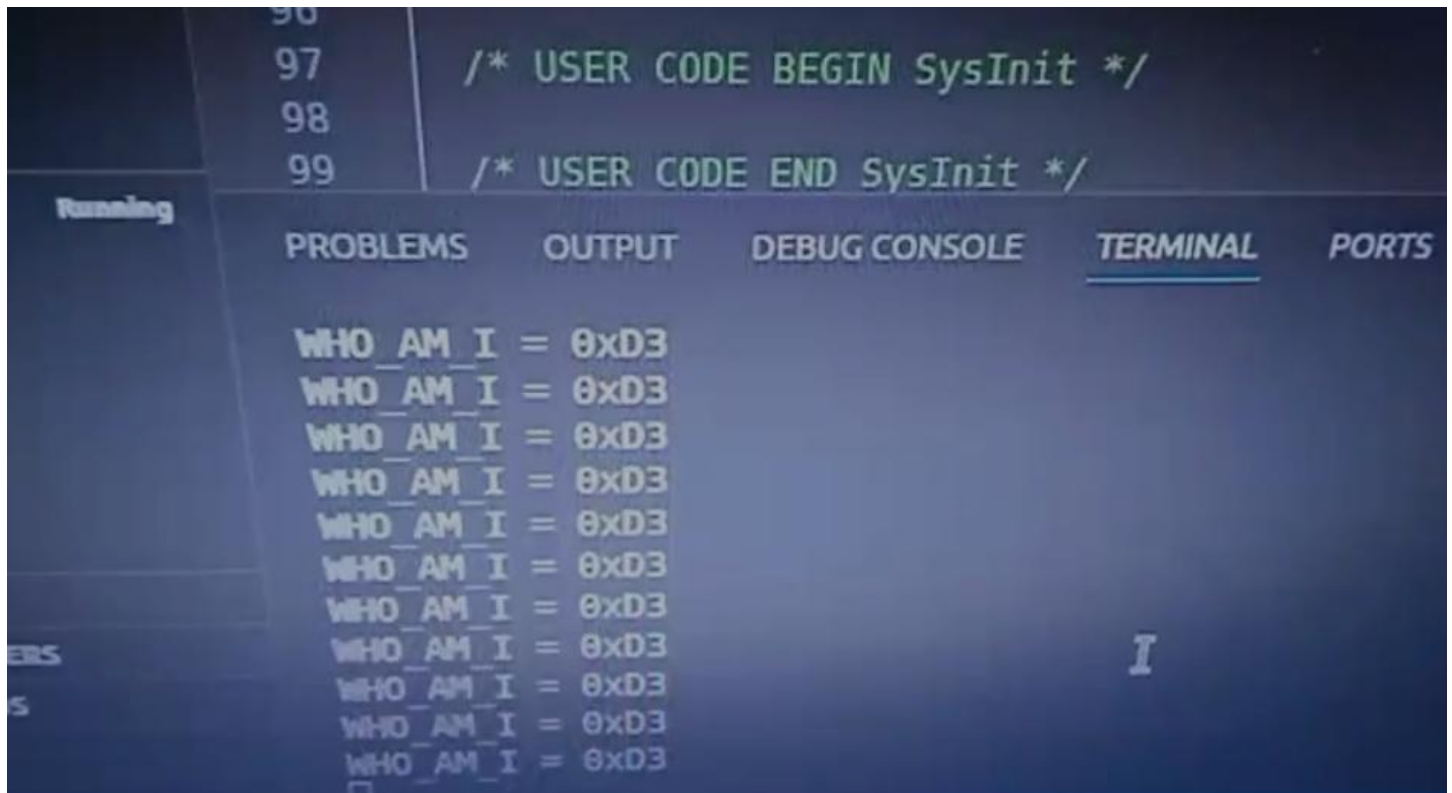
```
//   sprintf(uart_buf, "WHO_AM_I = 0x%02X --> L3GD20 (PASS)\r\n", who_am_i);
```

```
// else
```

```
//   sprintf(uart_buf, "WHO_AM_I = 0x%02X --> UNKNOWN\r\n", who_am_i);
```

```
// HAL_UART_Transmit(&huart2, (uint8_t*)uart_buf, strlen(uart_buf), 100);
```

RESULTS:



The screenshot shows an IDE interface with a terminal window. The terminal displays the output of a program. At the top, there is a code editor with the following C code:

```
96  
97  /* USER CODE BEGIN SysInit */  
98  
99  /* USER CODE END SysInit */
```

Below the code editor, the terminal window is active, showing the output of the program. The output consists of multiple lines of the text "WHO_AM_I = 0xD3". The terminal window has tabs for "PROBLEMS", "OUTPUT", "DEBUG CONSOLE", "TERMINAL", and "PORTS". The "TERMINAL" tab is selected. The output is as follows:

```
WHO_AM_I = 0xD3  
WHO_AM_I = 0xD3  
WHO_AM_I = 0xD3  
WHO_AM_I = 0xD3  
WHO_AM_I = 0xD3  
WHO_AM_I = 0xD3  
WHO_AM_I = 0xD3  
WHO_AM_I = 0xD3  
WHO_AM_I = 0xD3  
WHO_AM_I = 0xD3  
WHO_AM_I = 0xD3  
WHO_AM_I = 0xD3
```

TASK 2:

```
// typedef enum {  
//   SPI_STATE_IDLE = 0,  
//   SPI_STATE_TX_DONE,  
//   SPI_STATE_RX_DONE  
// } SPI_State_t;  
  
// volatile SPI_State_t spiState = SPI_STATE_IDLE;  
  
// uint8_t t2_tx_buf[1];  
  
// uint8_t t2_rx_buf[1];  
  
// volatile int8_t t2_temperature = 0;  
  
// void gyro_read_temp_start(void);  
  
// void HAL_SPI_TxCpltCallback(SPI_HandleTypeDef *hspi);  
  
// void HAL_SPI_RxCpltCallback(SPI_HandleTypeDef *hspi);  
  
// void gyro_read_temp_start(void)
```

```

//{
//  t2_tx_buf[0] = SPI_READ | OUT_TEMP;
//  spiState  = SPI_STATE_IDLE;
//  GYRO_CS_LOW();
//  HAL_SPI_Transmit_IT(&hspl1, t2_tx_buf, 1);
//}

//

// void HAL_SPI_TxCpltCallback(SPI_HandleTypeDef *hspl)
//{
//  if (hspl->Instance == SPI1)
//  {
//    spiState = SPI_STATE_TX_DONE;
//    HAL_SPI_Receive_IT(&hspl1, t2_rx_buf, 1);
//  }
//}

//

// void HAL_SPI_RxCpltCallback(SPI_HandleTypeDef *hspl)
//{
//  if (hspl->Instance == SPI1)
//  {
//    GYRO_CS_HIGH();
//    t2_temperature = (int8_t)t2_rx_buf[0];
//    spiState  = SPI_STATE_RX_DONE;
//  }
//}

// GYRO_CS_HIGH();

// HAL_Delay(10);

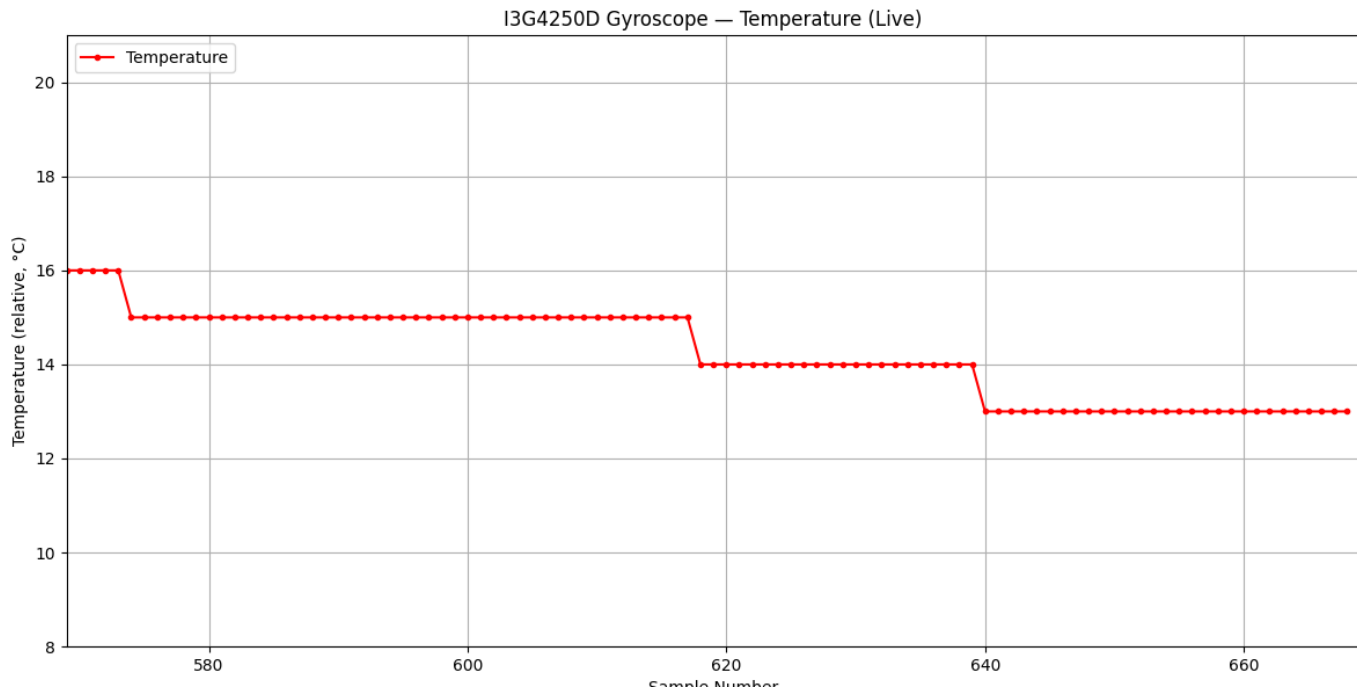
// gyro_init_t2(); /* Task 2 used its own init writing only CTRL_REG1=0x0F */

// HAL_Delay(100);

// -- while loop was: gyro_read_temp_start(); wait spiState; transmit; delay

```

RESULTS:



TASK 3:

```
static uint8_t rx_buf[6];

static int8_t temp_raw;

static int16_t x_raw, y_raw, z_raw;

static char  uart_buf[64];

/* USER CODE END PV */

void SystemClock_Config(void);

static void MX_GPIO_Init(void);

static void MX_I2C1_Init(void);

static void MX_SPI1_Init(void);

static void MX_USART2_UART_Init(void);

static void MX_USB_PCD_Init(void);

static void gyro_write_reg(uint8_t reg, uint8_t val);

static uint8_t gyro_read_reg(uint8_t reg);

static void gyro_init(void);
```

```

static void gyro_read_all(void);

static int write_dps(char *buf, int32_t raw);

static void gyro_write_reg(uint8_t reg, uint8_t val)
{
    uint8_t tx[2] = { reg & 0x3F, val };

    GYRO_CS_LOW();

    HAL_SPI_Transmit(&hspi1, tx, 2, HAL_MAX_DELAY);

    GYRO_CS_HIGH();
}

static uint8_t gyro_read_reg(uint8_t reg)
{
    uint8_t cmd = SPI_READ | (reg & 0x3F);

    uint8_t data = 0;

    GYRO_CS_LOW();

    HAL_SPI_Transmit(&hspi1, &cmd, 1, HAL_MAX_DELAY);

    HAL_SPI_Receive (&hspi1, &data, 1, HAL_MAX_DELAY);

    GYRO_CS_HIGH();

    return data;
}

static void gyro_init(void)
{
    gyro_write_reg(CTRL_REG1, CTRL_REG1_VAL); /* Power on, enable axes */
    gyro_write_reg(CTRL_REG4, CTRL_REG4_VAL); /* ±245 dps full scale */
}

static void gyro_read_all(void)
{
    /* Read temperature */

```

```

temp_raw = (int8_t)gyro_read_reg(OUT_TEMP);

/* Burst read 6 bytes: OUT_X_L through OUT_Z_H using auto-increment */
uint8_t cmd = SPI_READ | SPI_AUTO_INC | OUT_X_L; /* 0xE8 */
GYRO_CS_LOW();
HAL_SPI_Transmit(&hspi1, &cmd, 1, HAL_MAX_DELAY);
HAL_SPI_Receive (&hspi1, rx_buf, 6, HAL_MAX_DELAY);
GYRO_CS_HIGH();

/* Merge low and high bytes (little-endian) */
x_raw = (int16_t)((rx_buf[1] << 8) | rx_buf[0]);
y_raw = (int16_t)((rx_buf[3] << 8) | rx_buf[2]);
z_raw = (int16_t)((rx_buf[5] << 8) | rx_buf[4]);
}

static int write_dps(char *buf, int32_t raw)
{
    int32_t scaled = (raw * 875L) / 100L; /* units of 0.01 dps */
    int negative = (scaled < 0);
    if (negative) scaled = -scaled;

    int32_t whole = scaled / 100;
    int32_t frac = scaled % 100;

    if (negative)
        return sprintf(buf, "-%ld.%02ld", (long)whole, (long)frac);
    else
        return sprintf(buf, "%ld.%02ld", (long)whole, (long)frac);
}

int main(void)

```

```
{  
  
    /* USER CODE BEGIN 1 */  
  
    /* USER CODE END 1 */  
  
    HAL_Init();  
  
    SystemClock_Config();  
  
    MX_GPIO_Init();  
    MX_I2C1_Init();  
    MX_SPI1_Init();  
    MX_USART2_UART_Init();  
    MX_USB_PCD_Init();  
    GYRO_CS_HIGH();  
    HAL_Delay(10);  
    gyro_init();  
    HAL_Delay(100);  
  
    /* USER CODE END 2 */  
  
    /* USER CODE BEGIN WHILE */  
    while (1)  
    {  
        /* USER CODE END WHILE */  
  
        /* USER CODE BEGIN 3 */  
  
        gyro_read_all();  
  
        char xs[12], ys[12], zs[12];  
        write_dps(xs, x_raw);
```

```
write_dps(ys, y_raw);
```

```
write_dps(zs, z_raw);
```

```
int len = sprintf(uart_buf, "%d, %s, %s, %s\n",
```

```
    (int)temp_raw, xs, ys, zs);
```

```
HAL_UART_Transmit(&huart2, (uint8_t *)uart_buf, (uint16_t)len, HAL_MAX_DELAY);
```

```
HAL_Delay(100);
```

```
}
```

```
/* USER CODE END 3 */
```

```
}
```

RESULT:

```
17, 17.67,
-56.87, -10.67
17, 7.87, 3.15, 0.43
17, 1.05, -2.97, 0.35
17, -3.32, -4.46, 4.46
17, 3.
41, -2.45, 1.05
17, 1.13, -4.90, 6.65
17, -2.53, -1.75, -3.93
17, 1.40, -5.33, 2.01
18,
0.96, 0.52, -2.18
18, -0.70, -13.30, -6.03
18, 3.76, -4.63, -2.10
18, -0.26, 11.20, -2.
01
18, -1.57, -16.80, -1.48
18, 2.18, -4.81, -1.92
18, 0.08, 2.62, -1.05
18, 0.43, -11.2
0, 0.26
18, -3.85, -8.57, 3.93
18, 1.66, 5.95, -0.35
17, -2.01, -7.52, 3.23
17, -2.80, -
8.22, -1.05
17, -3.06, 7.35, -0.87
17, 0.52, -9.45, 2.45
17, 0.43, -10.41, 2.71
```

